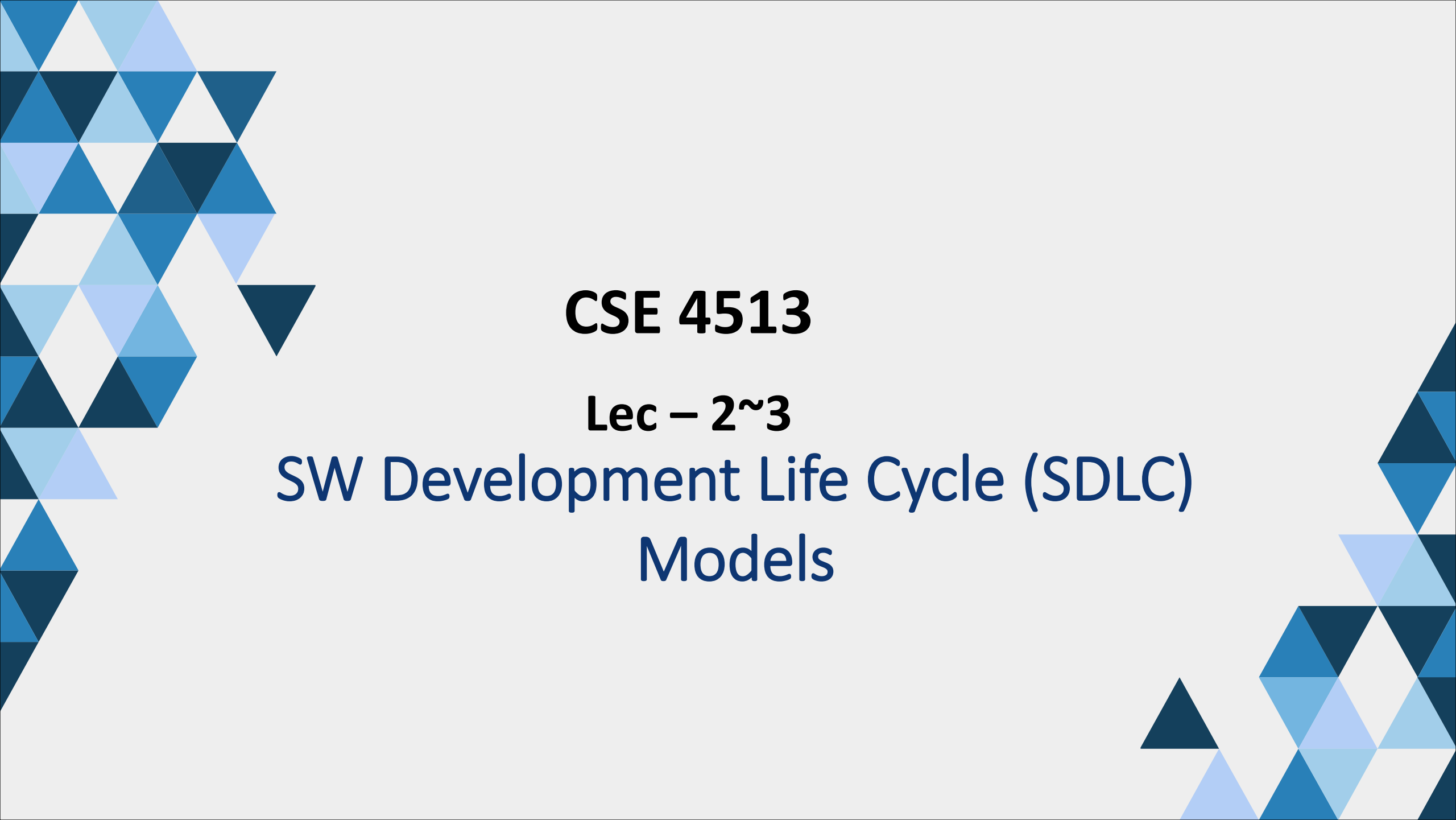




**CSE 4513**

**Lec – 2~3**

**SW Development Life Cycle (SDLC)  
Models**

# INHERENT PROBLEMS WITH SOFTWARE DEVELOPMENT



## ✓ **Requirements are complex**

what is functional requirements? - clarify

- The client **does not know the functional requirements** in advance

## ✓ **Requirements may be changing**

- Technology enablers introduce **new possibilities** to deal with nonfunctional requirements

## ✓ **Frequent changes are difficult to manage**

- **Identifying milestones** and **cost estimation** is difficult

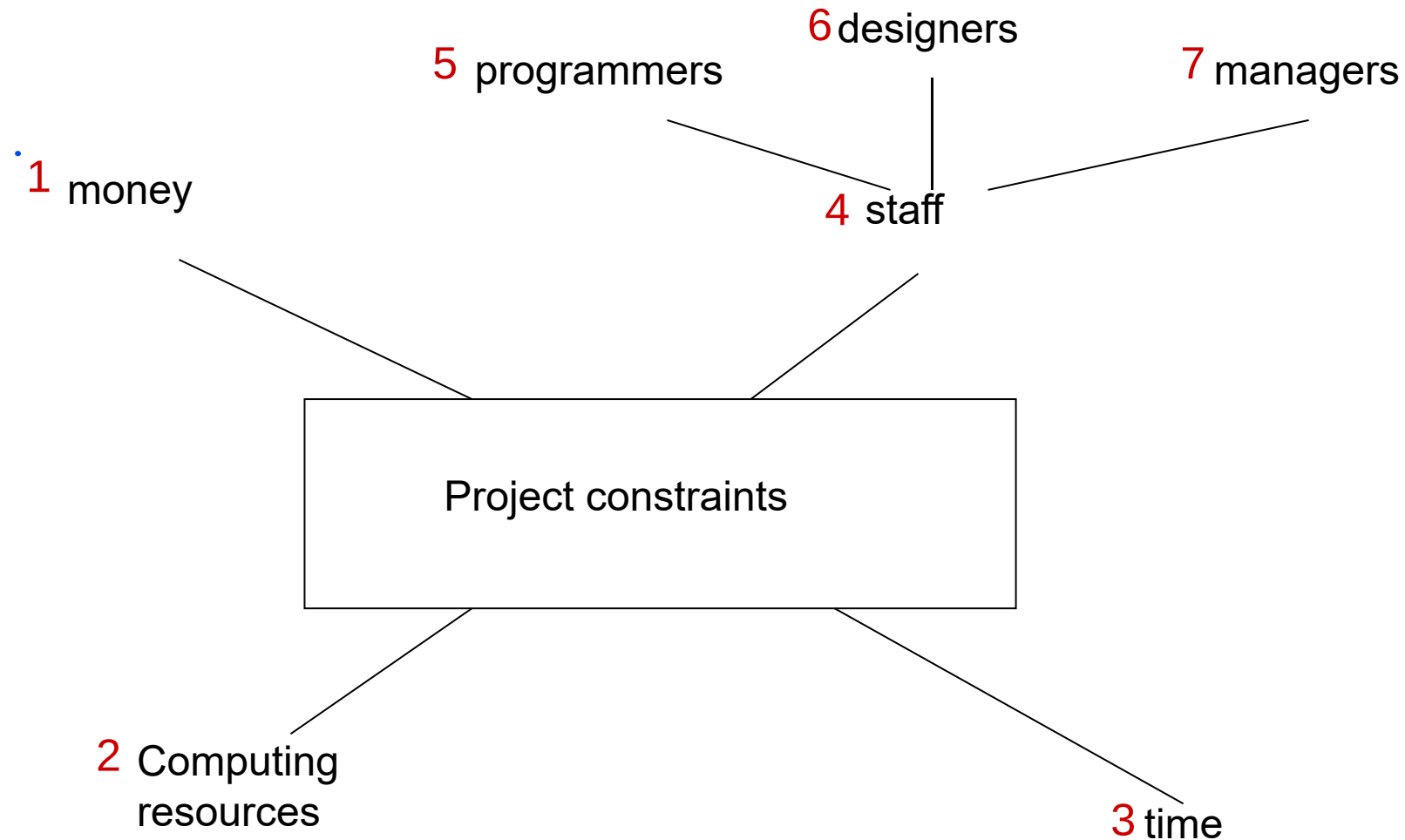
## ✓ **There is more than one software system**

- New system must be **backward compatible with existing system** (“legacy system”)

- ✍ **Phased development:** **Need to distinguish** between the **system under development** and **already released systems**

All these lead us to software life cycle modeling

# CONSTRAINTS OF SOFTWARE DEVELOPMENT PROJECT



## Examples of SW Project Constraints

# SW DEVELOPMENT LIFE CYCLE (SDLC) MODELS

---



Popular Software developing life cycles (SDLC) models:

- ✓ Waterfall Model
- ✓ V-Shaped Model
- ✓ Iterative Incremental Model
- ✓ Spiral Model
- ✓ Prototyping Model
- ✓ Agile development

# WATERFALL MODEL

---



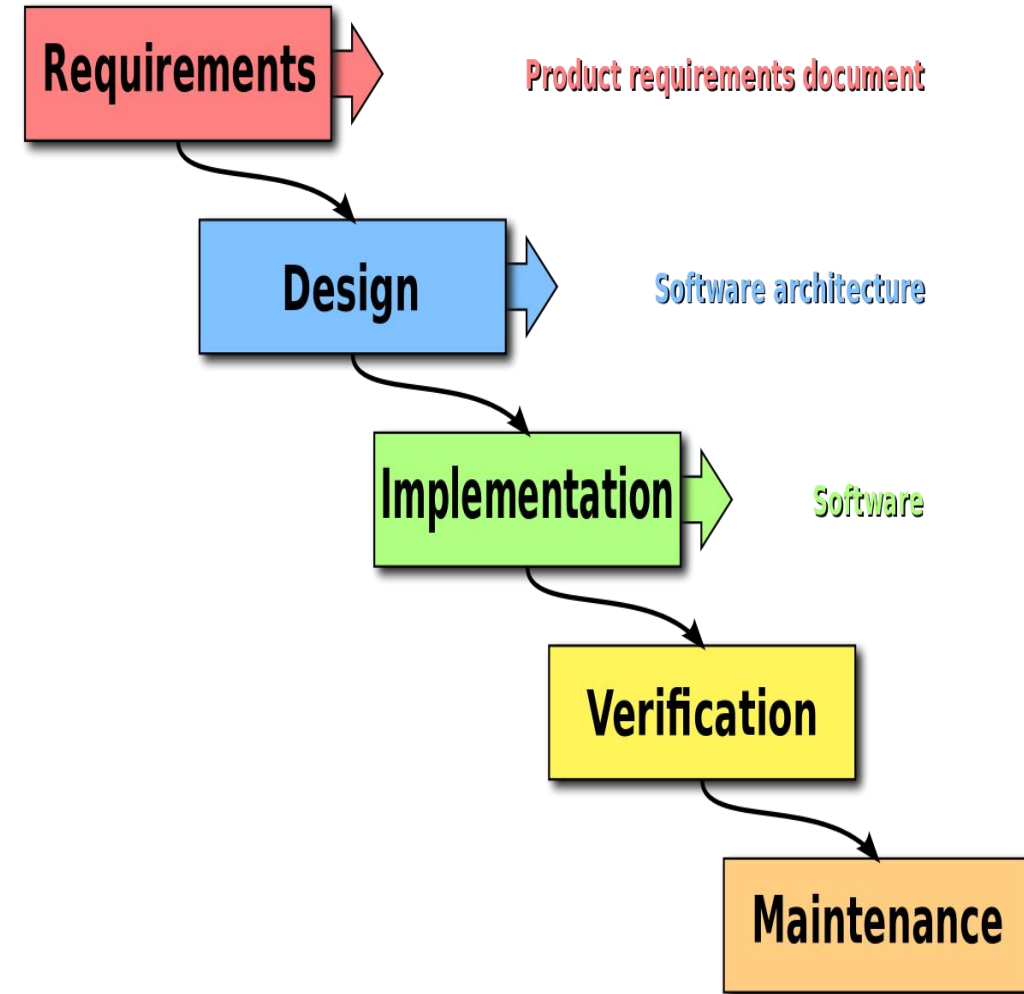
- ✓ The first formal description of the waterfall model is often cited as a 1970 article by Winston W. Royce.
- ✓ Royce presented this model as an example of a flawed, non-working model.
- ✓ It has been widely used for software projects ever since.

Royce, Winston (1970), "Managing the Development of Large Software Systems"

# WATERFALL MODEL



- ✓ It is also called as **linear sequential** model.
- ✓ In this model **whole application is developed in a sequential approach.**
- ✓ In this model **each phase must be completed fully** before the next phase begin.
- ✓ **does not define the process to go back** to the previous phase to handle changes in requirement.
- ✓ **Provides structure** to inexperienced staff.



# WHEN TO USE WATERFALL MODEL

---



List of things to consider when using the waterfall:

- ✓ The requirements are clear and frozen.
- ✓ Technology is understood and used by the team in different projects.
- ✓ The project cannot be delivered in an iterative manner.
- ✓ Documentation is essential.
- ✓ Professional Project management skills.
- ✓ The project cost is defined.

# ✓ WATERFALL MODEL - ADVANTAGES



- ✓ It is very easy to explain to the business users and explain the output of each phase.
- ✓ **Structured approach.**
- ✓ Stages and activities are **well defined.**
- ✓ It is easier for project managers to plan, schedule the project, utilize the resources, and define the milestones easily.
- ✓ **Validation** and **verification at each phase** ensure early detection of errors or misunderstanding at the same phase.
- ✓ Each phase has **specific deliverables.**



# ✓ WATERFALL MODEL - DISADVANTAGES

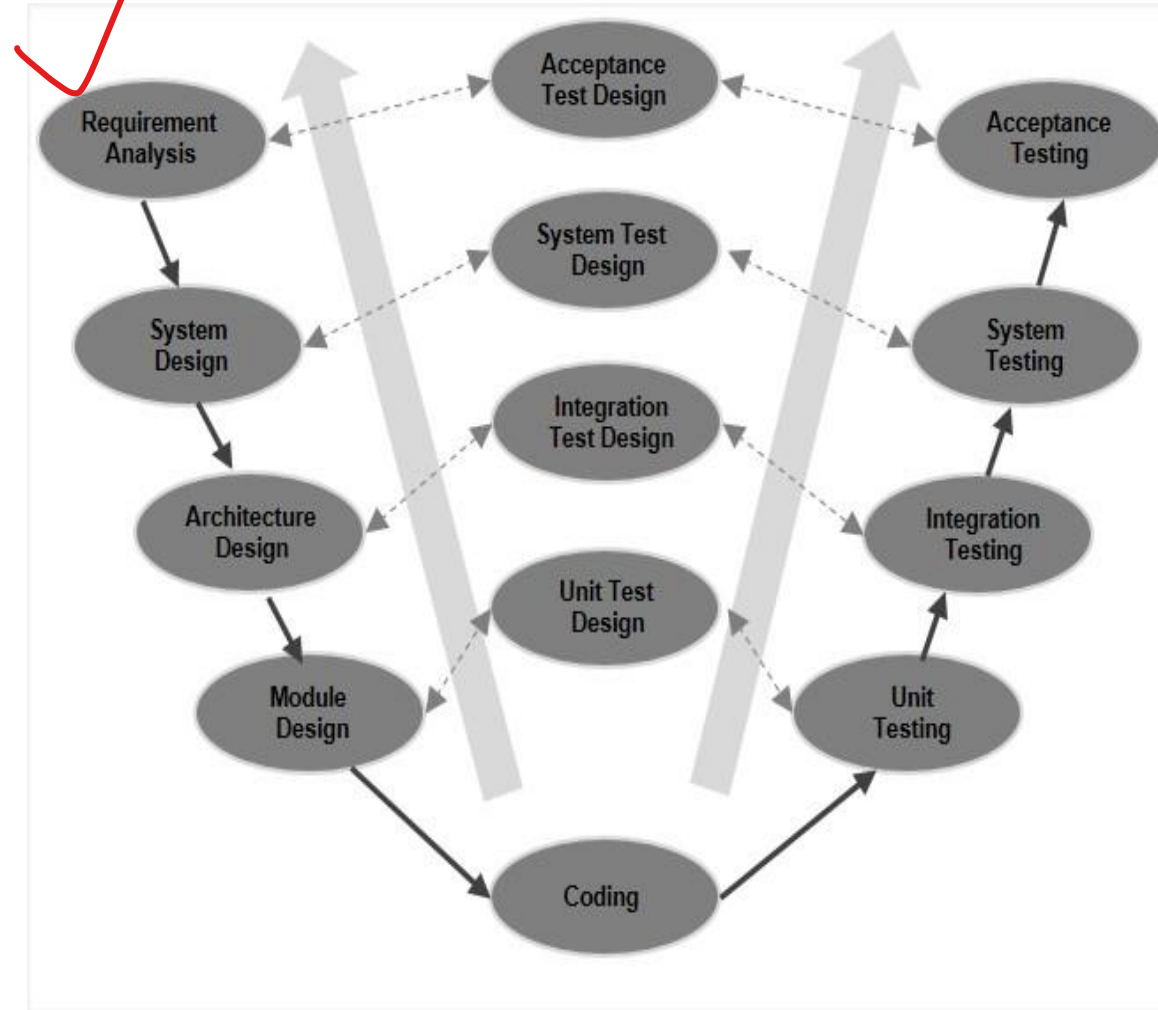


- ✓ It takes the full lifecycle to deliver a workable solution to the customer.
- ✓ It is very difficult to go back to any phase after it finished.
- ✓ It assumes that the requirements of a system can be frozen without any changes or enhancements.
- ✓ A little flexibility and adjusting scope is difficult and expensive.
- ✓ It requires more time for the detailed plan upfront of the project, as the requirements are clear and frozen and it should be visible to have the detailed plan delivered to the customer.
- ✓ It delays the testing phase which can discover a lot of issues in requirements, design, and implementation as well.

# V-SHAPED MODEL



- ✓ It is an extension of the waterfall model, Instead of moving down in a linear way, the process steps are bent upwards. after the implementation and coding phase, to form the typical V shape.
- ✓ The major difference between the V-shaped model and waterfall model is the early test planning in the V-shaped model.



# WHEN TO USE V MODEL

---



List of things to consider when using the V model:

- ✓ Requirements are well defined, clearly documented and fixed.
- ✓ Product definition is stable.
- ✓ Technology is not dynamic and is well understood by the project team.
- ✓ There are no ambiguous or undefined requirements.
- ✓ The project is short in nature

# V MODEL - ADVANTAGES



- ✓ Simple and easy to use
- ✓ Each phase has specific deliverables.
- ✓ Higher chance of success over the waterfall model due to the development of test plans early on during the life cycle.
- ✓ Works well for where requirements are easily understood.
- ✓ Verification and validation of the product in the early stages of product development.

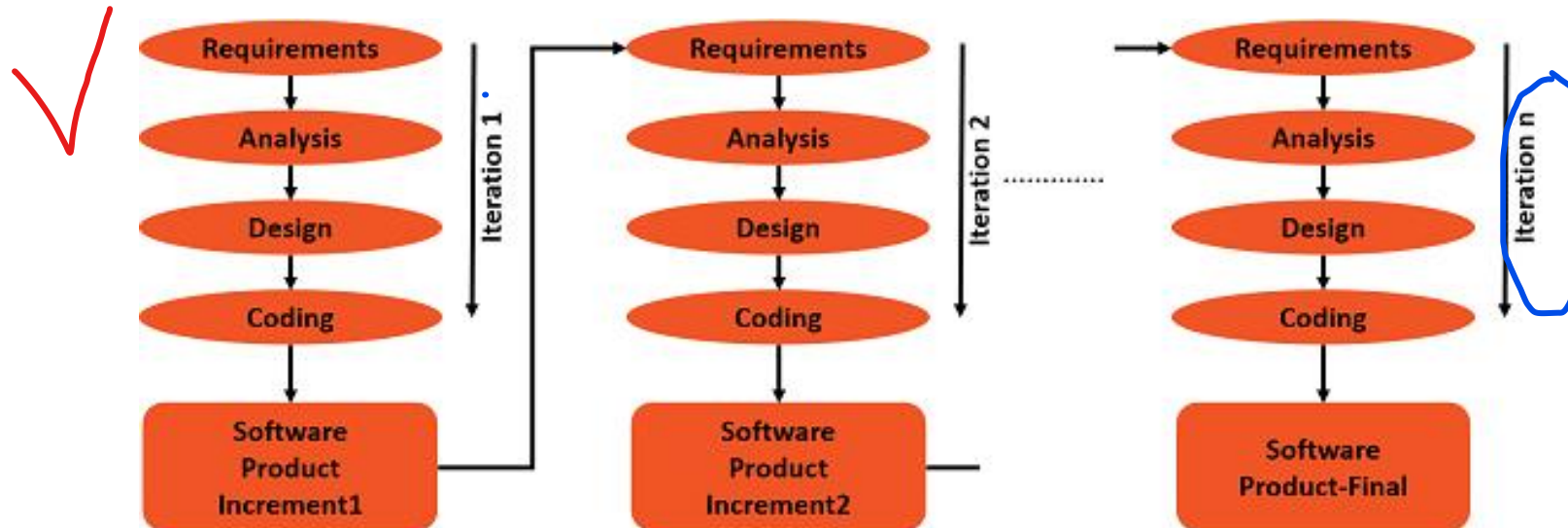
# V MODEL - DISADVANTAGES

- ✓ Very inflexible, like the waterfall model.
- ✓ Adjusting scope is difficult and expensive.
- ✓ The software is developed during the implementation phase, so no early prototypes of the software are produced.
- ✓ The model doesn't provide a clear path for problems found during testing phases.
- ✓ Costly and required more time, in addition to a detailed plan

# ITERATIVE INCREMENTAL MODEL



- ✓ developed to overcome the weaknesses of the waterfall model.
- ✓ starts with an initial planning and ends with deployment with the cyclic interactions in between.
- ✓ The basic idea behind this method is to develop a system through repeated cycles (iterative) and in smaller portions at a time (incremental).
- ✓ allowing software developers to take advantage of what was learned during the development of earlier parts or versions of the system.
- ✓ It can consist of mini waterfalls or mini V-Shaped model



# WHEN TO USE ITERATIVE & INCREMENTAL METHOD

---



List of things to consider :

- ✓ Most of the requirements are known up-front but are expected to evolve over time.
- ✓ The requirements are prioritized.
- ✓ There is a need to get the basic functionality delivered fast.
- ✓ A project has lengthy development schedules.
- ✓ A project has new technology.
- ✓ The domain is new to the team.

# ITERATIVE INCREMENTAL MODEL - ADVANTAGES



- ✓ You can develop prioritized requirements first.
- ✓ Initial product delivery is faster.
- ✓ Customers gets important functionality early.
- ✓ Low initial delivery cost. ✖
- ✓ Each release is a product increment, so that the customer will have a working product at hand all the time.
- ✓ Customer can provide feedback to each product increment, thus avoiding surprises at the end of development.
- ✓ Requirements changes can be easily accommodated.



# ITERATIVE INCREMENTAL MODEL - DISADVANTAGES



- ✓ Requires effective planning of iterations.
- ✓ Requires efficient design to ensure inclusion of the required functionality and provision for changes later.
- ✓ Requires early definition of a complete and fully functional system to allow the definition of increments.
- ✓ Well-defined module interfaces are required, as some are developed long before others are developed.

# SPIRAL MODEL



- ✓ is a combination of a waterfall model and iterative model.
- ✓ Each phase in spiral model begins with a design goal and ends with the client reviewing the progress.
- ✓ The development team in Spiral-SDLC model starts with a small set of requirement and goes through each development phase for those set of requirements.
- ✓ The software engineering team adds functionality for the additional requirement in every-increasing spirals until the application is ready for the production phase

# SPIRAL MODEL



## Spiral “areas”

### Planning

includes estimating the cost, schedule and resources for the iteration. also involves understanding the system requirements for continuous communication between the system analyst and the customer.

### Evaluation

Evaluation of software by the customer. Also, includes identifying and monitoring risks such as schedule slippage and cost overrun

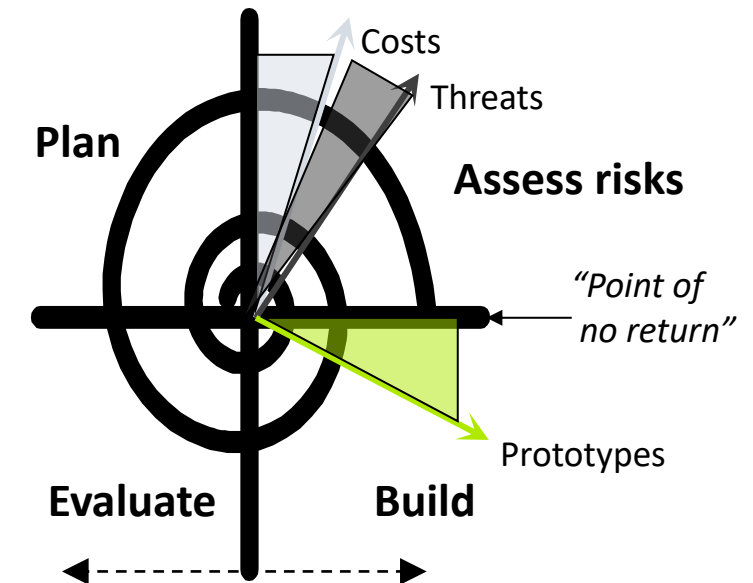
### Risk analysis

Identification of potential risk is done while risk mitigation strategy is planned and finalized

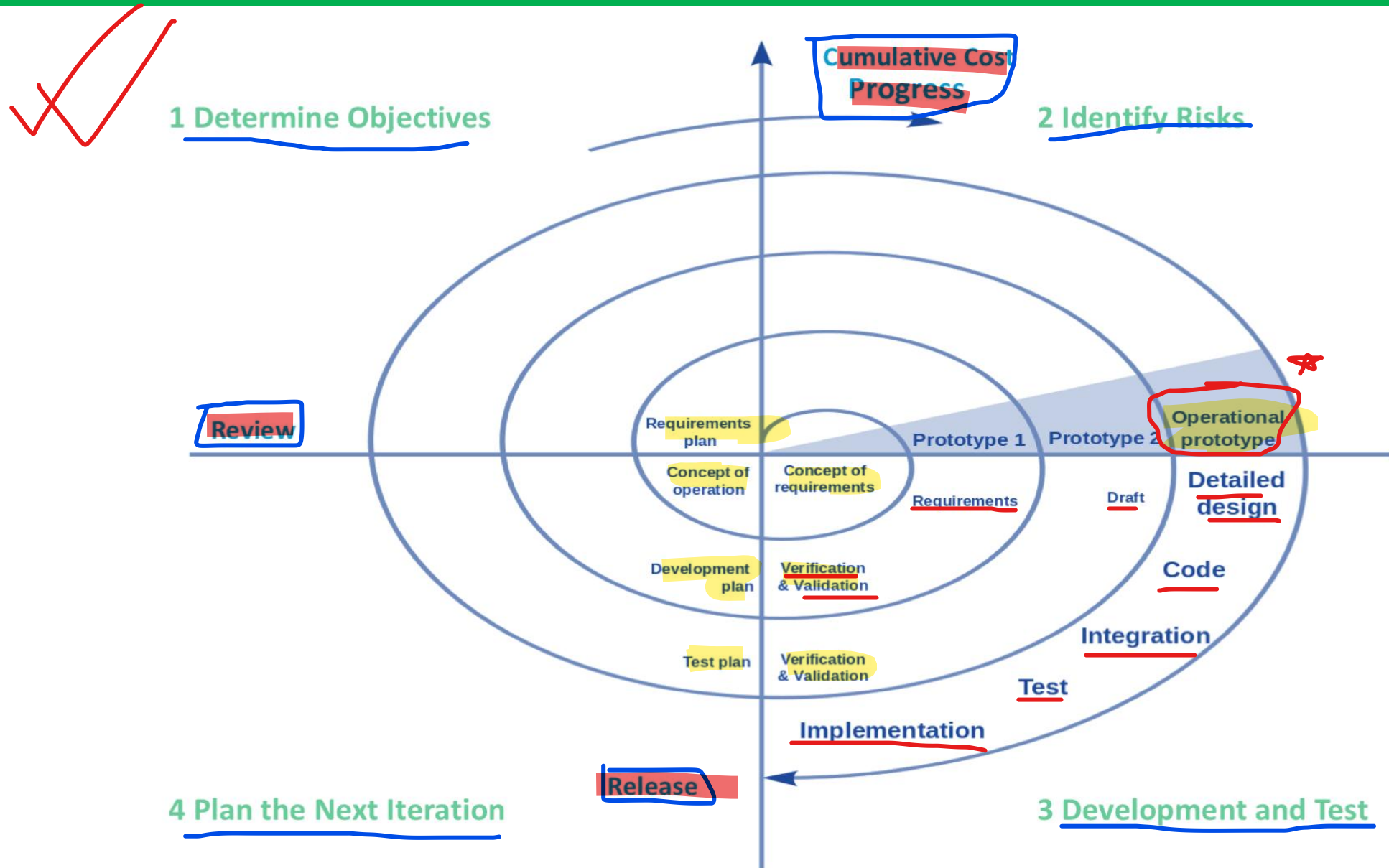
### Engineering

includes testing, coding and deploying software at the customer site

✓ Diagrammatically said...



# SPIRAL MODEL



# SPIRAL MODEL - STEPS ۷۱



- 1) Define requirements
  - Through user involvement and analysis of existing system
- 2) Initial new system design
- 3) Construct and evaluate an initial prototype
  - Rough (skeletal) system attribute framework
- 4) Construct a further (refined) prototype
  - Basing it on evaluation of initial prototype
  - Defining its scope
  - Planning its development
  - Implementing it
- 5) Overall (system-wide) risk assessment
- 6) **Prototype assessment** (as per step 4) and possible development of **further prototypes**
- 7) Repeat steps 1-5 until **refined prototype meets user expectations**
- 8) **Construct the system** (based on final refined prototype)
- 9) **Test and maintain** the system

# WHEN TO USE SPIRAL MODEL



List of things to consider when using the Spiral Model:

- ✓ When **project is large**
- ✓ When **releases are required to be frequent**
- ✓ When creation of a prototype is applicable
- ✓ When **risk and costs evaluation** is important
- ✓ For medium to high-risk projects
- ✓ When **requirements are unclear** and complex
- ✓ When **changes may require** at any time

# SPIRAL MODEL - ADVANTAGES



- ✓ Additional functionality or changes can be done at a later stage
- ✓ Cost estimation becomes easy as the prototype building is done in small fragments
- ✓ Continuous or repeated development helps in risk management
- ✓ Development is fast and features are added in a systematic way
- ✓ There is always a space for customer feedback

# SPIRAL MODEL - DISADVANTAGES



- ✓ Risk of not meeting the schedule or budget
- ✓ It works best for large projects only also demands risk assessment expertise
- ✓ For its smooth operation spiral model protocol needs to be followed strictly
- ✓ Documentation is more as it has intermediate phases
- ✓ It is not advisable for smaller project, it might cost them a lot



# PROTOTYPING MODEL



“It is easier to tell what you don’t like about an existing system than to describe what you would like in an imaginary one”

A.M. Jenkins, 1983

★  
is the process of quickly putting together a **working model** (a prototype) in order to test various aspects of a design, illustrate ideas or features and gather early user feedback

IEEE defines prototyping as “ A type of development in which emphasis is placed on developing prototypes early in the development process to permit early feedback and analysis in support of the development process.”

# PROTOTYPING MODEL - TYPES



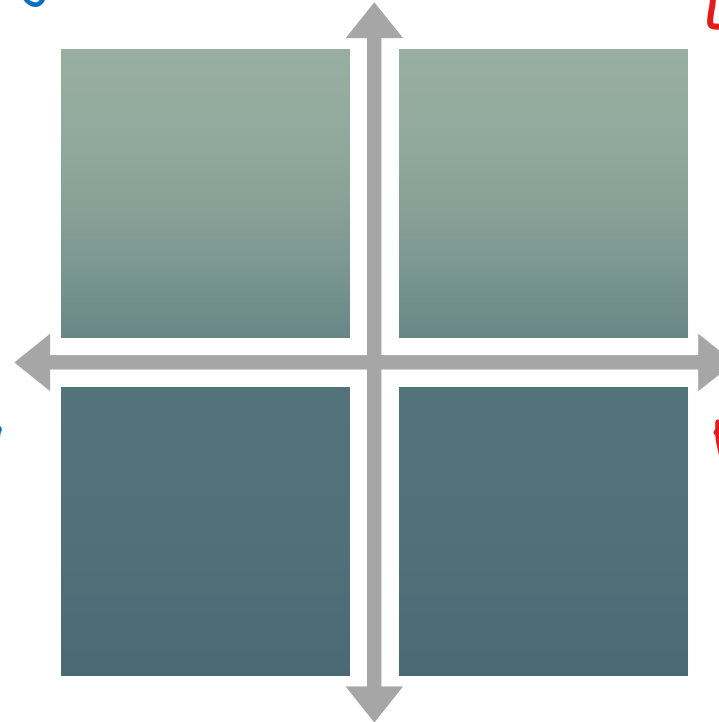
## Major software prototyping types

### Throwaway/Rapid Prototyping

- uses very little efforts with **minimum requirement analysis** to build a prototype.
- Once the actual requirements are understood, the prototype is discarded and the actual system is developed with a much clear understanding of user requirements.

### Incremental Prototyping

- Incremental prototyping refers to building **multiple functional prototypes of the various sub-systems** and then **integrating all the available prototypes** to form a complete system.



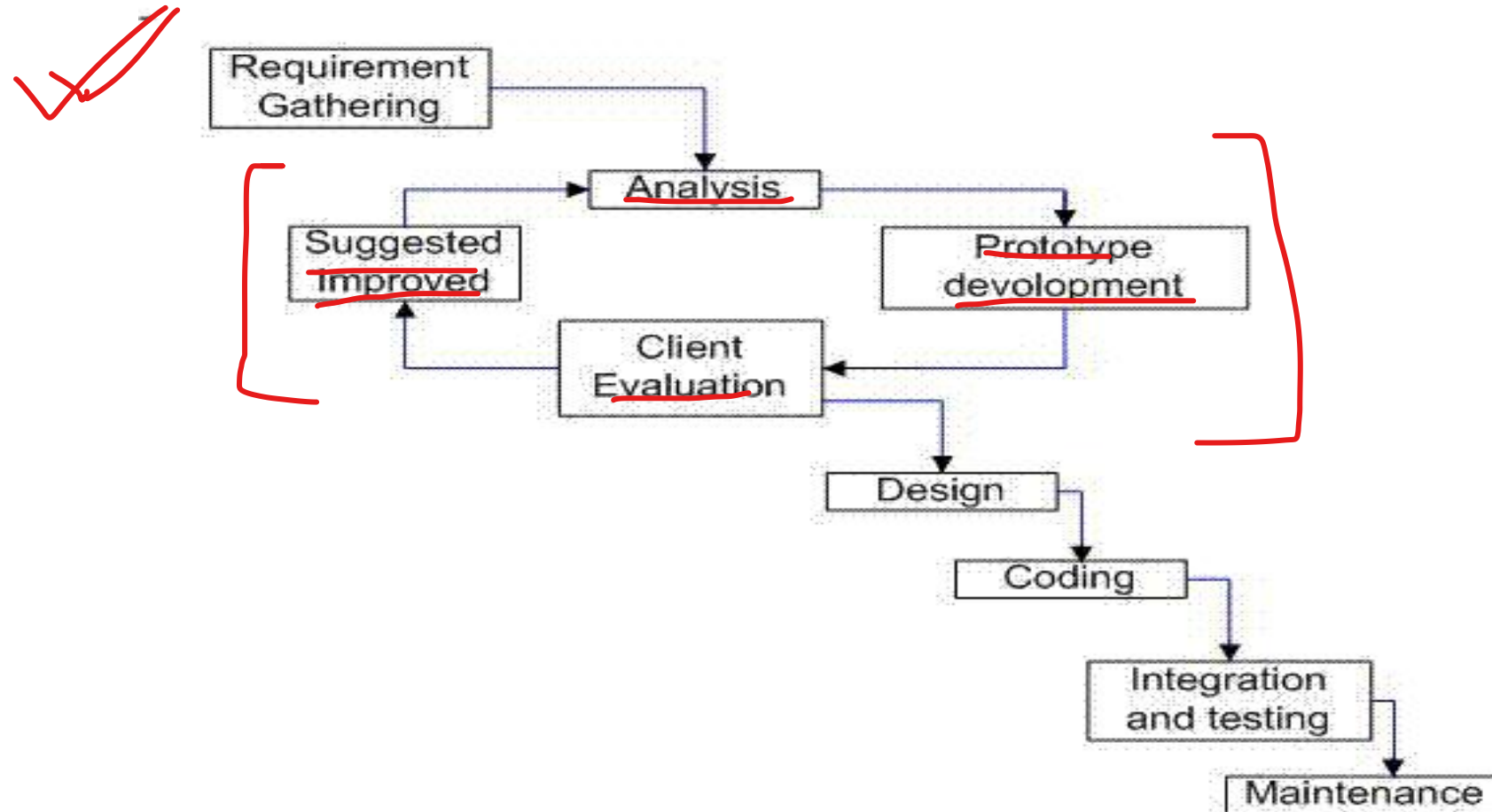
### Evolutionary Prototyping

- is based on building **actual functional prototypes** with **minimal functionality** in the beginning.
- Only the well-understood requirements are included in the prototype and the requirements are added as and when they are understood.

### Extreme Prototyping

- is used in the **web development domain**.
- consists of **three sequential phases**.
  - First, a basic prototype with all the existing pages is presented in the HTML format.
  - Then the data processing is simulated using a **prototype services** layer.
  - Finally, the services are implemented and integrated to the final prototype.

# PROTOTYPING MODEL



**Evolutionary Prototyping Model**

# WHEN TO USE PROTOTYPING MODEL



## List of things to consider when using the Prototyping Model:

- ✓ the requirements are unclear
- ✓ if requirements are changing quickly.
- ✓ can be successfully used for developing user interfaces, high technology software-intensive systems, and systems with complex algorithms and interfaces.
- ✓ a very good choice to demonstrate the technical feasibility of the product.

# PROTOTYPING MODEL - ADVANTAGES



- ✓ errors can be detected in the initial stage of the software development process.
- ✓ Missing functionality can be identified, which helps to reduce the risk of failure
- ✓ Customer satisfaction exists because the customer can feel the product at a very early stage.
- ✓ As customers are involved from the early stage, there will be hardly any chance of software rejection.
- ✓ Quicker user feedback helps you to achieve better software development solutions.
- ✓ It is a straightforward model, so it is easy to understand.
- ✓ No need for specialized experts to build the model
- ✓ The prototype serves as a basis for deriving a system specification.
- ✓ The prototype helps to gain a better understanding of the customer's needs.
- ✓ Prototypes may offer early training for future users of the software system.

# PROTOTYPING MODEL - DISADVANTAGES



- ✓ There may be **too much variation in requirements** each time the prototype is evaluated by the customer.
- ✓ **Poor Documentation** due to continuously changing customer requirements.
- ✓ It is very difficult for the developers to **accommodate all the changes** demanded by the customer.
- ✓ There is uncertainty in **determining the number of iterations** that would be required before the prototype is finally accepted by the customer.
- ✓ After seeing an early prototype, the customers sometimes demand the **actual product to be delivered soon**.
- ✓ Developers in a hurry to build **prototypes** may end up with **sub-optimal solutions**.
- ✓ The customer might **lose interest** in the product if he/she is not satisfied with the initial prototype.

# PROTOTYPING MODEL



Do we need prototyping??



Two “points of interest” for companies to adopt prototyping based methodologies are:

**Point 1:** They allow us to **reduce the cost** and **time-to-market** of a system.

**Point 2:** For companies building critical systems, prototyping would help them perform **formal verification** when required. These methodologies provide high level of **reliability** in the system design and implementation.

scenario: company that has a tested prototype is more reliable than company that doesn't